



Time to read: 6–8 minutes

Next topic: *How AI systems are built differently from traditional software*

Big idea: In AI systems, data is part of the code — and deployment is the beginning.

1) Why compare SDLCs?

Most learners already understand **traditional SDLC** (requirements → design → build → test → deploy → maintain).

AI projects also follow a lifecycle, but the *source of behaviour* is different:

- In traditional software, **behaviour comes from rules written in code**.
- In AI systems, **behaviour is learned from data** and encoded inside a model.

That single difference changes everything: testing, maintenance, monitoring and release strategy.

2) Traditional SDLC (What you already know)

Traditional software has characteristics that make it stable and testable.

Traditional SDLC characteristics (explained)

Deterministic logic

The logic is explicit: if/else, rules, functions.

Same code + same inputs = same output.

Example:

If password matches → login success. If not → login denied.

Stable requirements (relatively)

Requirements can change, but changes are:

- Documented
- Planned

- Implemented through code updates

The system doesn't "silently change itself."

Bugs are reproducible

When something breaks:

- You capture logs
- Replay the scenario
- Reproduce the bug
- Fix it
- Ship an update

This is the classic debug loop: **reproduce → isolate → fix → validate**.

Deployment is the end (or close to it)

After deploying, the system is expected to behave consistently until the next release.

Monitoring focuses mostly on:

- Uptime
- Errors
- Performance

3) AI SDLC (Why AI needs a different lifecycle)

AI systems behave differently because they are **probabilistic** and **data-dependent**.

AI SDLC characteristics (explained)

Probabilistic behaviour

AI models output:

- Predictions
- Probabilities
- Rankings
- Generated text

Even small changes to the input can produce different outputs or different confidence scores.

Example:

Two customers who look similar might get different risk scores because the model learned complex patterns.

Data defines behaviour

In AI the “logic” is not fully in the code.

It is embedded in:

- The training data
- The labels
- Feature engineering choices
- Model architecture and weights

Key implication:

You can keep the same code and still change system behaviour by changing the data.

Models degrade over time.

The real-world changes:

- User behaviour changes
- New fraud patterns emerge
- Customer preferences shift
- Sensors drift
- New products roll out

So performance can drop without any code changes.

This is not a bug in the code—it’s a property of learning systems.

Continuous monitoring is needed.

Because AI can degrade silently, you must monitor:

- Data drift
- Prediction distribution shifts
- Delayed performance (when labels arrive)
- Latency and cost
- Business impact signals

Monitoring is not “nice to have.” It is part of the design.

Deployment is the beginning.

In AI, deployment starts a new phase:

- Collecting real-world feedback
- Detecting drift
- Iterating and retraining
- Improving performance safely

Traditional software: Deployment ends the build cycle.

AI systems: Deployment begins the improvement cycle.

4) The Key Insight (memorise this)

In AI systems, data is part of the code.

What does that really mean?

In traditional software:

- Changing behaviour requires changing code.

In AI systems:

- Behaviour can change if the data changes
- Even if no developer touched the code

So, the engineering discipline must expand to manage:

- Data lifecycle
- Model lifecycle
- Monitoring
- Versioning and rollback
- Governance and compliance

5) Side-by-side comparison (for quick clarity)

Traditional SDLC

AI SDLC

Behaviour defined by code Behaviour learned from data + model

Deterministic outputs Probabilistic outputs

Bugs reproducible Failures may be silent or gradual

Testing checks correctness Testing checks performance + robustness

Deployment is the end Deployment is the beginning

Monitoring: uptime/errors Monitoring: drift + quality + cost + safety

6) A simple example to make it obvious

Example: Spam detection

If built as traditional software (rule-based)

- Rule: If subject contains “FREE MONEY” → spam
- Easy to test: same input → same result
- Stable behaviour until rules change

If built as AI (ML-based)

- The model learns patterns from labelled emails
- Spammers change patterns over time
- Performance drops unless you monitor and retrain

This is why MLOps exists.

7) Practical takeaway (engineering mindset)

When building AI systems, you must design for:

- Change
- Uncertainty

- Monitoring
- Safe iteration
- Operational control

That's why this course focuses on **systems**, not just models.

Bridge to the next topic

Now that you understand the difference between SDLCs, the next step is:

AI-specific SDLC phases (problem → data → model → deploy → monitor → iterate)

and how tooling supports each phase.